



Penetration Testing

Libertum 4th (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	19
Our Methodology	20
Disclaimers	22
About Hashlock	23

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Libertum team partnered with Hashlock to conduct a security Penetration Test of their Libertum Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Libertum is a licensed end-to-end tokenization engine for issuing, managing, and servicing compliant security tokens. This includes a comprehensive token suite and marketplaces - all white labelled for scalability

The Institutional Infrastructure for the Age of Digital Finance

Libertum are building the backbone of the tokenized economy — enabling compliant, scalable, and liquid real-world asset markets. Our modular infrastructure combines T-Suite, a licensed tokenization and asset-servicing engine, with B-DEX, an AI-powered liquidity and yield platform that bridges traditional finance and decentralized ecosystems.

Project Name: Libertum

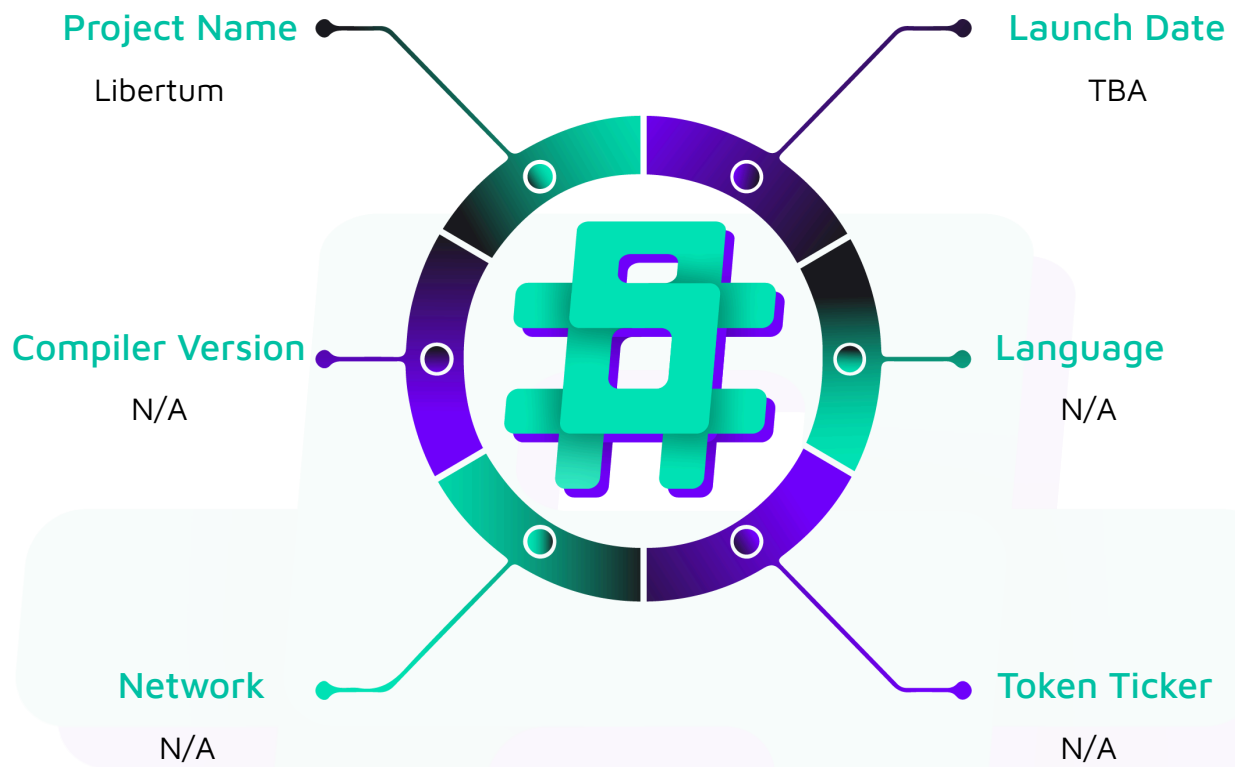
Project Type: Digital Asset, RWA, Security Tokens, RWAfi, TradFi, DeFi

Compiler Version: ^0.8.17

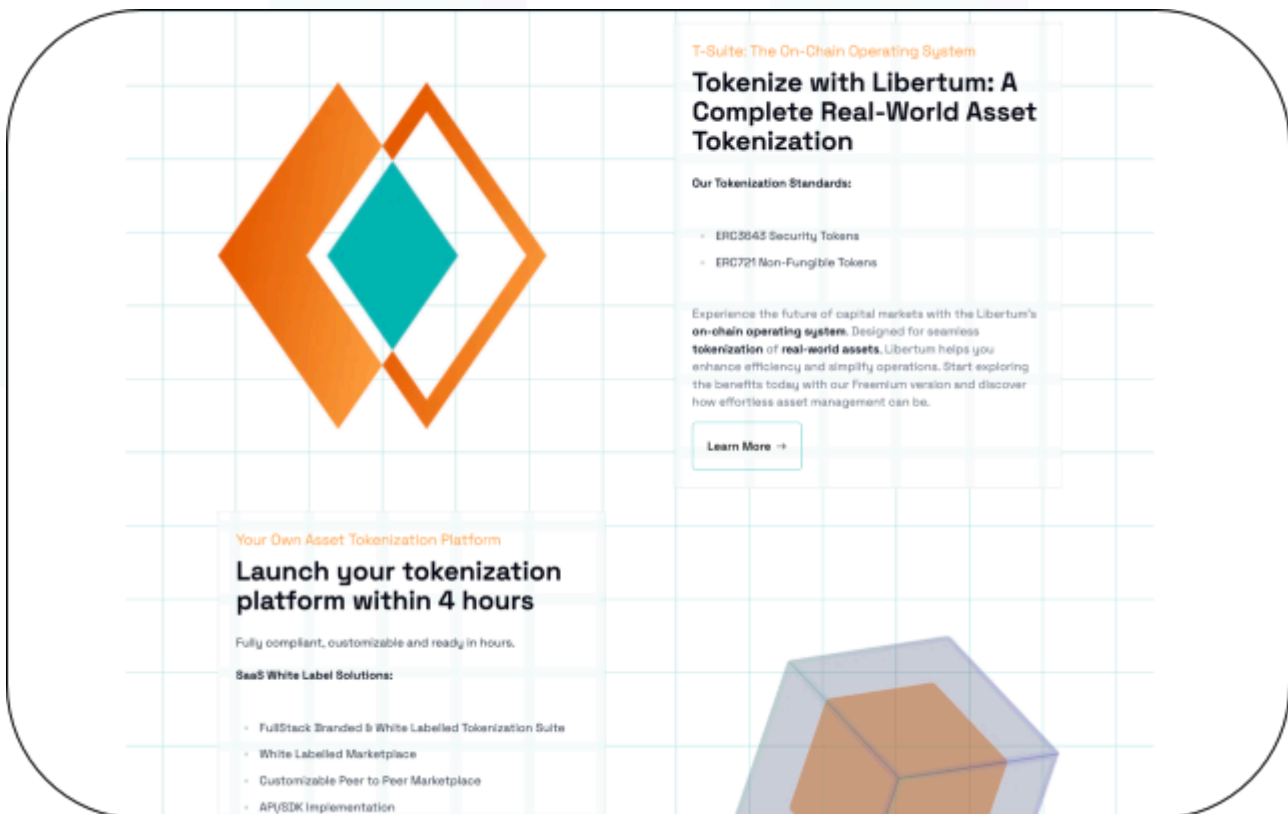
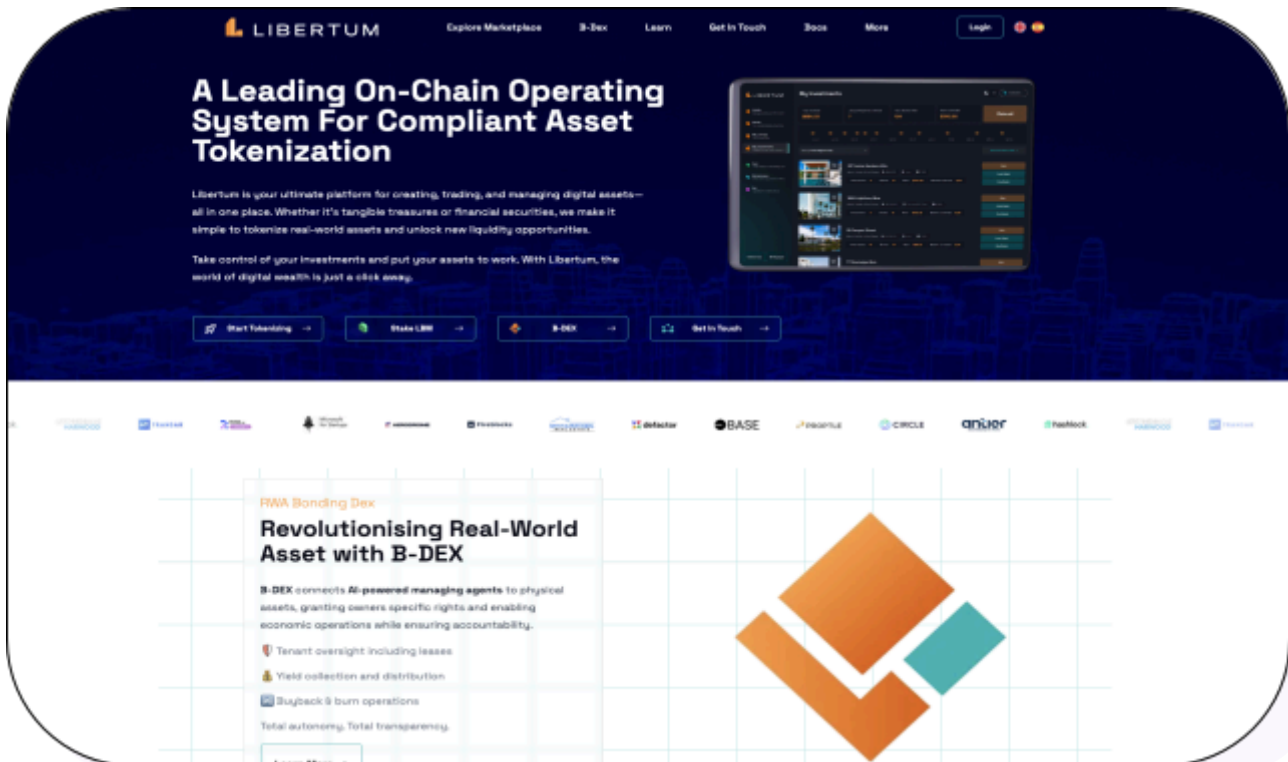
Website: <https://www.libertum.io/en/>

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Libertum project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Libertum Project Code Base
Platform	Ethereum / Solidity/Web apps & APIs
Penetration Test Date	October, 2025
Websites	Sandbox Admin https://sandbox-admin.libertum.io/ Sandbox Investor https://sandbox.libertum.io/ Sandbox Issuer https://sandbox.libertum.io/ Sandbox TA https://sandbox-ta.libertum.io/

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



A secure badge reflects that the client has successfully passed the audit.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved or acknowledged.

Hashlock found:

2 Medium-severity vulnerabilities

1 Low-severity vulnerability

1 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

Medium

[M-01] sandbox-api.libertum.io - Email enumeration

Description

The `/marketplace/v1/verify` and `/marketplace/v1/forgot-password` endpoints reveal whether an email address is registered in the system through differences in response behavior. When processing OTP verification requests, the endpoint returns distinct responses for registered versus unregistered email addresses, allowing attackers to enumerate valid user accounts.

Vulnerability Details

The `verify` endpoint at `/marketplace/v1/verify` accepts POST requests containing `userName` (email). The endpoint processes these requests, causing the application to return different responses based on whether the supplied email address exists in the user database. An attacker can submit requests with arbitrary email addresses and invalid OTPs, then observe the response variations to determine account existence.

```
POST /marketplace/v1/verify HTTP/1.1

Host: sandbox-api.libertum.io

Content-Type: application/json

{
  "userName": "[redacted]@gmail.com",
  "type": "sign_up",
  "otp": "378446",
  "otpMethods": "Email"
}
```

Additionally, in `/marketplace/v1/forgot-password` endpoint exposes registered email addresses through differences in rate limit responses. Although the endpoint returns a generic message stating "Please Verify the Email Address Entered. If it is Registered, an OTP will be Sent to You," submitting the same email twice reveals account existence through the message "You can Request OTP again in 55 Seconds" which appears only if a code was successfully sent.

Impact

Attackers can enumerate all registered user accounts. This enables the same privacy violations and targeted attack scenarios as direct enumeration, including phishing campaigns and account takeover attempts.

Recommendation

Apply rate limiting to all password reset requests regardless of email validity. Login, forgot password, and confirm endpoints can implement CAPTCHA for maximum security.

Status

Resolved

[M-02] All subdomains - Missing rate limiting

Description

The application lacks rate-limiting controls across all API endpoints, allowing attackers to submit unlimited requests without throttling. This enables denial of service attacks and amplifies the email enumeration vulnerabilities.

Impact

Attackers can execute denial of service attacks by flooding the API with requests, degrading performance for legitimate users, or causing service outages.

The absence of rate limiting also accelerates email enumeration attacks from hours to minutes, allowing compilation of the complete user database.

Recommendation

Implement Cloudflare WAF rate limiting rules to restrict requests per IP address and per endpoint.

Status

Resolved

Low

[L-01] Contracts - Missing HTTP headers

Description

The application does not implement critical HTTP security headers, leaving users vulnerable to clickjacking, content sniffing attacks, and man-in-the-middle exploitation. The missing headers include Content Security Policy, HTTP Strict Transport Security, and X-Content-Type-Options.

Vulnerability Details

Response headers lack the following security controls:

- Content-Security-Policy (CSP): No CSP header exists, allowing execution of inline scripts and loading resources from any origin. The frame-ancestors directive is absent, permitting the application to be embedded in iframes on malicious sites.
- Strict-Transport-Security (HSTS): The HSTS header is not configured, failing to enforce HTTPS connections and allowing potential SSL stripping attacks.
- X-Content-Type-Options: Missing the nosniff directive, which allows browsers to MIME-sniff responses and potentially execute malicious content disguised as benign file types.

While this is a generic hardening recommendation, users remain vulnerable to clickjacking attacks where the application is embedded in malicious iframes to capture credentials. The absence of HSTS allows network attackers to downgrade connections to HTTP and intercept sensitive data. Missing content-type protection enables MIME confusion attacks where uploaded files execute as scripts.

Recommendation

Configure the web server to include security headers in all responses. Set

```
Content-Security-Policy: default-src 'self'; frame-ancestors 'none',
Strict-Transport-Security: max-age=31536000; includeSubDomains, and
```

X-Content-Type-Options: nosniff. Implement these headers at the CDN or reverse proxy level for consistent application across all endpoints.

Status

Resolved



QA

[Q-01] *.libertumio - Subdomains found in OSINT activities

Description

Besides the provided scope, using OSINT tools it was possible to reveal more publicly available subdomains:

```
www.libertum.io
app.libertum.io
assets.libertum.io
beta.libertum.io
beta-admin.libertum.io
beta-admin-api.libertum.io
beta-admin-ta.libertum.io
beta-api-ta.libertum.io
beta-dividend-api.libertum.io
beta-frontend-admin.libertum.io
beta-marketplace-api.libertum.io
beta-ta.libertum.io
blog.libertum.io
bonding.libertum.io
bookings.libertum.io
decap.libertum.io
dev-admin.libertum.io
dev-website.libertum.io
docs.libertum.io
earn.libertum.io
```



www.marketplace.libertum.io
marketplace.libertum.io
qa.libertum.io
qa-admin.libertum.io
qa-api.libertum.io
qa-ta.libertum.io
sandbox.libertum.io
stage.libertum.io
stage-admin-api.libertum.io
stage-admin-ta.libertum.io
stage-api-ta.libertum.io
stage-bonding.libertum.io
stage-bonding-api.libertum.io
stage-frontend-admin.libertum.io
stage-marketplace-api.libertum.io
staging-admin.libertum.io
beta-bonding-api.libertum.io
qa-bonding.libertum.io
sandbox-admin.libertum.io
sandbox-admin-api.libertum.io
sandbox-api-ta.libertum.io
sandbox-bonding.libertum.io
sandbox-bonding-api.libertum.io
sandbox-dividend-api.libertum.io
sandbox-marketplace-api.libertum.io
sandbox-ta.libertum.io

```
stage-dex.libertum.io
stage-dex-api.libertum.io
stage-dividend-api.libertum.io
forms.libertum.io
dex.libertum.io
dex-api.libertum.io
libertum.io
qa-dex.libertum.io
qa-dex-api.libertum.io
sandbox-api.libertum.io
admin-staging.libertum.io
admin-dev.libertum.io
admin.libertum.io
marketplace-staging.libertum.io
marketplace-test.libertum.io
www.test-admin.libertum.io
```

Recommendation

None. This is for information only. The subdomains should be verified to determine if they should be publicly available.

Status

Acknowledged

Conclusion

After Hashlock's analysis, the Libertum project seems to have a sound and well-tested code base, however, now that our vulnerability findings have been resolved or acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.